

Faculty of Engineering and Technology

Department of Electrical and Computer Engineering

Operating System-ENCS3390

Project Report

Prepared by:

Student Name: Mayar Etawi. **Student Number: 1230501.**

Instructor: Abdel Salam Sayyad.

Section: 1

Date: 04/11/2025

Table of Contents

My Environment Description 4
Computer Specs 4
Virtual Machine Specs 4
Programming Language and IDE tool 5
Multiprocessing Implementation and Used APIs 6
Multithreading Implementation and Used APIs 6
Result 7
Multiprocessing Results 8
Multithreading Results 9
Analysis According to Amdahl's Law 10
The Serial Part percentage of the Code 10
Theoretical Maximum Speedup 10
Optimal Number of Threads or Processes 10
Comparison the performance of the three approaches &Discussion 11
Conclusion 13

Table of Table

Table 1: Numerical Results..... 7
Table 2: Speedup Results Multiprocessing..... 8
Table 3:Speedup Results Multithreads. 8
Table 4: The Execution time & Performance for Multiprocessing..... 11
Table 5: The Execution time & Performance for Multithreading..... 12

Table of Figure

Figure 1: Virtual Machine Settings part-1. Figure 2: Virtual Machine Settings Part-2. 5
Figure 3: Virtual Machine - system Details. 5
Figure 4: Sample result. 7

My Environment Description

Computer Specs

I implemented this project using a computer with the following specifications:

Processor

Intel Core i5-1135G7 11th Generation processor, which contains 4 cores, 8 logical cores (threads). The processor base frequency is 2.40 GHz, and has a 8 MB cache, with Bus speed of 4 GT/s. It is x64-based processor

Memory

The Memory type is DDR4-3200, 8 GB RAM with speed of 3200MT/s. The max Memory bandwidth is 45.8 GB/s, Memory Channel 2.

Operating system

The Operating system of my device is Windows 11,64-bit based.

Virtual Machine Specs

In this project, I used VirtualBox to set up a virtual machine running the Ubuntu Linux operating system, and I implemented and tested the program within this environment.

I allocate 4 cores, 4 logical core (threads) to this virtual machine and 4096 MB RAM from the Memory.

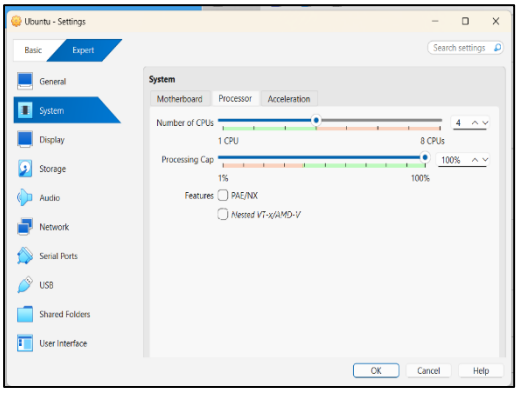


Figure 1: Virtual Machine Settings part-1.

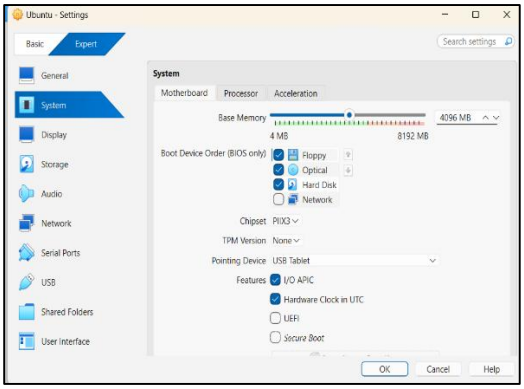


Figure 2: Virtual Machine Settings Part-2.

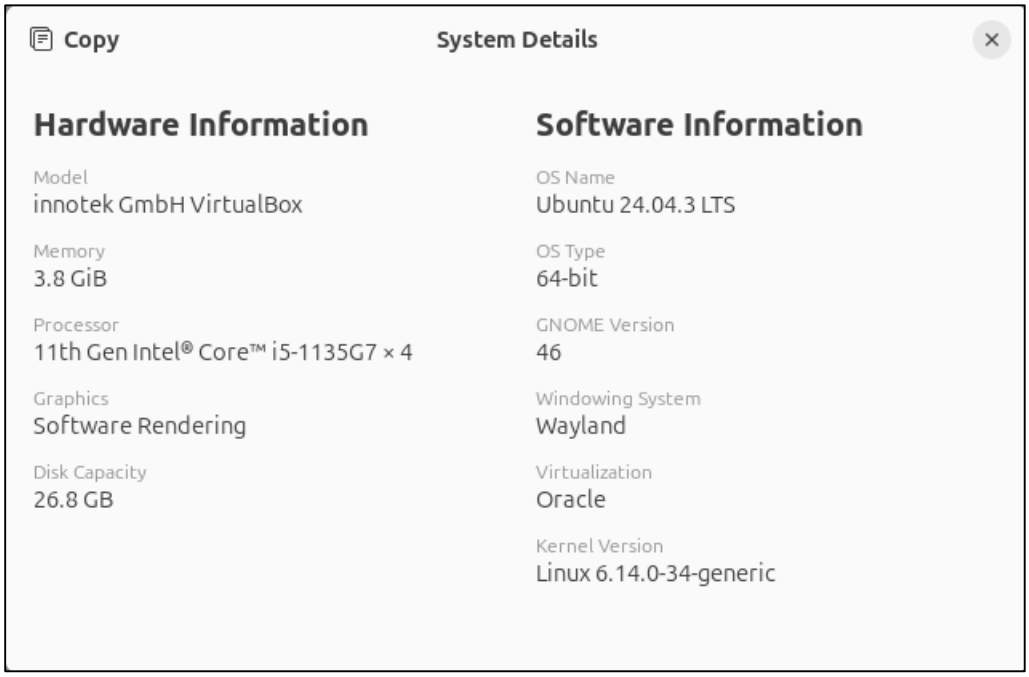


Figure 3: Virtual Machine - system Details.

Programming Language and IDE tool

I developed this project using the C programming language on Ubuntu (Linux), running inside VirtualBox.

The development, I use the Code::Blocks IDE, which relies on the GCC compiler to build the project and run the executable inside the Ubuntu virtual machine.

Multiprocessing Implementation and Used APIs

The multiprocessing implementation was achieved using Unix process-control functions provided by <unistd.h> and <sys/wait.h>. After reading all records into memory, the dataset is divided into **20 segments**, matching the number of child processes to be created. For each segment, the program creates a unidirectional communication pipe using pipe() and then calls fork() to generate a child process.

Each child process closes the read end of the pipe and processes its assigned range of records. Just like in the threaded version, the filtering logic relies on strcasecmp() to match the user-provided value. The child accumulates its own local results inside a ProcResult structure and sends this structure back to the parent using the write() system call. Once finished, the child closes its pipe and exits using _exit() to ensure a clean termination.

The parent process, meanwhile, keeps the list of child PIDs and closes the write end of all pipes. After all children have been created, the parent receives each partial result via read(), merges it into global totals, and uses waitpid() to ensure every child process has terminated correctly. Through the use of process-level APIs (fork, pipe, read, write, waitpid), the program performs true multiprocessing, where each process runs independently with its own memory space and contributes to the final combined output.

Multithreading Implementation and Used APIs

The multithreaded implementation was completed using the POSIX Threads API (pthreads) provided by the <pthread.h> library. After loading the 1-million sales records into a global dynamic array, the program divides the dataset into **16 equal segments**, corresponding to the predefined number of threads. Each thread is created using the function pthread_create(), which launches the worker function threadFunc() and assigns to it a specific portion of the dataset.

To perform the filtering, each thread receives a ThreadData structure that specifies its start index, end index, the filter type (region, country, item type, or sales channel), and the filter value provided by the user. Inside the worker function, the thread scans its assigned section and uses strcasecmp() to perform a case-insensitive comparison between the record field and the search value. Each thread updates only its local counters (totalOrders, totalRevenue, totalProfit), making the design thread-safe without requiring locks or mutexes since no shared data is modified.

After all threads finish execution, the main thread uses pthread_join() to wait for them and then merges their partial results into a final accumulated output. By using the pthread API (pthread_create, pthread_join) and distributing the dataset into independent slices, the program successfully achieves true multithreading and parallel data filtering.

Result

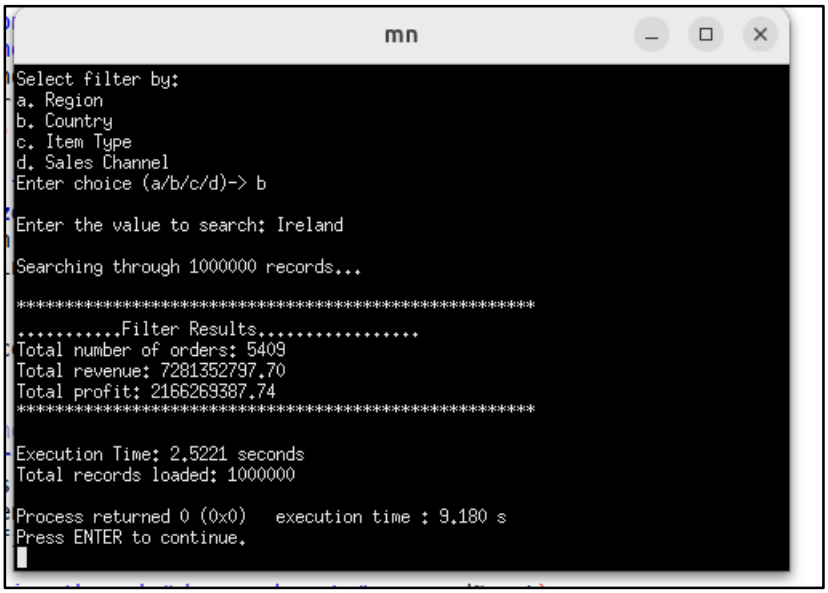


Figure 4: Sample result.

Table 1: Numerical Results

Case	Attribute	Instance	Total Orders	Total Revenue	Total Profit
I	Region	e.g. Europe	259411	345922520783.34	101930686144.31
II	Country	e.g. Ireland	5409	7281352797.70	2166269387.74
III	Item Type	e.g. Clothes	83123	45432507052	30532241196
IV	Sales Channel	e.g. Online	500140	664552299300.13	196009323723.93

Theoretically Speedup = $\frac{1}{S+\frac{1-S}{N}}$

Practically Speedup = $\frac{Excecution\ Time\ old}{Excecution\ Time\ new}$

Table 2: Speedup Results Multiprocessing.

Case	Naïve			2 child			4 child			8 child			16 child			20 child		
	Time	Theo speedup	Pra speedup	Time	Theo Speedup	Pra speedup	Time	Theo speedup	pra speedup	Time	Theo Speedup	Pra speedup	Time	Theo speedup	Pra speedup	Time	Theo speedup	pra Speedup
I	2.6149	3.72	1	0.9875	1.9519	2.648	0.9934	3.72	2.632	0.8656	3.72	3.02	0.9850	3.72	2.65	0.9624	3.72	2.71
II	2.3681	3.72	1	0.8147	1.9519	2.907	0.9134	3.72	2.593	0.9854	3.72	2.40	0.8550	3.72	2.76	0.9544	3.72	2.48
III	2.0430	3.72	1	0.9245	1.9519	2.209	0.8644	3.72	2.363	0.8457	3.72	2.41	1.0187	3.72	2.01	0.9777	3.72	2.09
IV	2.3162	3.72	1	0.8745	1.9519	2.649	0.9228	3.72	2.509	0.9078	3.72	2.55	1.0521	3.72	2.20	0.9766	3.72	2.37

Table 3:Speedup Results Multithreads.

Case	Naïve			2 Threads			4 Threads			8 Threads			16 Threads			20 Threads		
	Time	Theo speedup	Pra speedup	Time	Theo Speedup	Pra speedup	Time	Theo speedup	pra speedup	Time	Theo Speedup	Pra speedup	Time	Theo speedup	Pra speedup	Time	Theo speedup	pra Speedup
I	2.6149	3.72	1	0.9729	1.9519	2.68	0.8640	3.72	3.02	1.0398	3.72	2.51	1.1508	3.72	2.27	1.1544	3.72	2.264
II	2.3681	3.72	1	0.9146	1.9519	2.58	0.8580	3.72	2.76	0.885	3.72	2.67	0.9664	3.72	2.45	0.9778	3.72	2.42
III	2.0430	3.72	1	0.9116	1.9519	2.24	0.8985	3.72	2.27	0.9780	3.72	2.088	1.0772	3.72	1.89	0.9209	3.72	2.21
IV	2.3162	3.72	1	1.0825	1.9519	2.14	0.8891	3.72	2.61	0.9196	3.72	2.52	0.9214	3.72	2.51	1.0357	3.72	2.24

Multiprocessing Results

In analyzing the **multiprocessing** results (Table 2), we observe that the **theoretical speedup** remains constant at **3.72** across all cases. However, in some cases, such as with **2 processes**, we see that the **practical speedup** exceeds the **theoretical speedup**. For example, in **Case I**, the practical speedup reached 2.648, which is higher than the theoretical limit. This may seem surprising at first because the theoretical speedup represents the maximum possible gain. However, in reality, this can happen due to **system-specific factors**, such as **memory management** or **synchronization effects** between processes, which can cause the practical results to surpass theoretical expectations in certain cases.

However, once the number of processes is increased to 4 or more, **resource contention** and the effects of **context switching** begin to negatively impact performance, causing the practical speedup to decrease compared to the theoretical speedup.

Multithreading Results

Similarly, in **multithreading** (Table 3), we observe that the **theoretical speedup** remains constant at **3.72**. However, for **2 threads**, we see that the **practical speedup** in certain cases, such as **Case II**, was greater than the theoretical speedup, reaching 2.58. This suggests that in some situations, the system might have performed better than expected with fewer threads. This could be because, with a lower number of threads, the system experiences **less complexity** in synchronization, allowing it to utilize the CPU resources more efficiently.

Nevertheless, when the number of threads is increased to 4 or more, **practical speedup** gradually decreases, reflecting the negative impact of **thread synchronization** and resource contention.

In both **multiprocessing** and **multithreading**, we observe cases where the **practical speedup** for **2 processes or threads** exceeded the **theoretical speedup**. This suggests that in some cases, the system may **utilize resources more efficiently** or benefit from **unexpected optimizations** due to system-specific factors. However, beyond 4 processes or threads, **resource contention** and the effects of **context switching** start to affect performance, leading to **practical speedup diminishing** compared to the theoretical speedup.

Analysis According to Amdahl's Law

The Serial Part percentage of the Code:

To determine the serial portion of the program, we measured the execution time of the non-parallel section using the clock() function. The total execution time was recorded at the beginning and end of the program, while an additional pair of timestamps was taken before and after the menu-processing phase. This phase represents the part of the code that cannot be parallelized. The difference between these two measurements (ends – starts) was used to calculate the serial execution time, and dividing it by the total program time provided the percentage of the serial part. This method allowed us to experimentally quantify the sequential fraction of the program for further analysis under Amdahl's law.

Serial Percentage = $\frac{\text{SerialTime}}{\text{TotalTime}} * 100\% = \frac{0.0264}{2.4047} * 100\% = 1.0979\ \%$

Theoretical Maximum Speedup

The theoretical maximum speedup of the system is determined by the number of physical CPU cores available in the virtual machine. Since the VM is configured with **4 CPU cores**, the highest possible speedup—regardless of the number of threads or processes used—cannot exceed the number of cores. Therefore, the theoretical maximum speedup of our parallel implementation is:

$Speedup_{max}=4$

This represents the upper performance limit imposed by the hardware, assuming ideal conditions with no overhead or bottlenecks.

Optimal Number of Threads or Processes

The optimal number of threads or processes is determined by the number of physical CPU cores available on the system. Since the virtual machine used in this project provides **4 CPU cores**, the best performance is achieved when using **4 threads or 4 processes**. Increasing the number of threads or processes beyond this limit does not improve execution time; instead, it introduces additional overhead due to resource contention and increased context switching.

The experimental results support this behavior, as both multiprocessing and multithreading show their best practical speedup when approaching the 4-core limit. However, **multiprocessing achieved slightly higher maximum practical speedup than multithreading**, demonstrating better performance in this experiment, especially at 8 processes where the speedup peaked before declining.

Therefore, the optimal configuration for parallel execution in this environment is **4 processes or 4 threads**, with multiprocessing showing a marginal performance advantage over multithreading under the tested conditions.

Comparison the performance of the three approaches &Discussion

By applying three different approaches in our program — the Naïve method, Multiprocessing, and Multithreading — we were able to evaluate how each technique affects execution time and overall performance when calculating the total number of orders, revenue, and profit from the sales records. The results clearly show that introducing parallelism, whether through multiple processes or multiple threads, significantly reduces execution time compared to the sequential approach, since the workload is divided across several processing units instead of running in a single path. The performance differences observed between multiprocessing and multithreading reflect the way each method handles memory and system resources. The following table presents a detailed comparison of the execution time and performance achieved by each approach in our program.

$$\text{Performance} = \frac{1}{\text{Execution time}}$$

Table 4: The Execution time & Performance for Multiprocessing.

Case	Naïve		2 child		4 child		8 child		16 child		20 child	
	Time	Performance	Time	performance	Time	Performance	Time	performance	Time	performance	Time	performance
I	2.6149	0.382	0.9875	1.013	0.8640	1.007	0.8656	1.155	0.9850	1.015	0.9624	1.039
II	2.3681	0.422	0.8147	1.227	0.8580	1.095	0.9854	1.015	0.8550	1.169	0.9544	1.048
III	2.0430	0.489	0.9245	1.082	0.8985	1.157	0.8457	1.182	1.0187	0.982	0.9777	1.023
IV	2.3162	0.431	0.8745	1.142	0.8891	1.084	0.9078	1.102	1.0521	0.950	0.9766	1.024

Table 5: The Execution time & Performance for Multithreading.

Case	Naïve		2 Threads		4 Threads		8 Threads		16 Threads		20 Threads	
	Time	performance	Time	performance	Time	performance	Time	performance	Time	performance	Time	performance
I	2.6149	0.382	0.9729	1.028	0.9934	1.007	1.0398	0.961	1.1508	0.869	1.1544	0.866
II	2.3681	0.422	0.9146	1.093	0.9134	1.095	0.8850	1.129	0.9664	1.035	0.9778	1.023
III	2.0430	0.489	0.9116	1.097	0.8644	1.157	0.9780	1.022	1.0772	0.928	0.9209	1.086
IV	2.3162	0.431	1.0825	0.924	0.9228	1.084	0.9196	1.087	0.9214	1.085	1.0357	0.966

In the Naïve approach, operations are performed sequentially, meaning no utilization of multiple cores or processors. This results in poor performance, as it relies on a single core, which significantly slows down execution compared to the other approaches. For example, in Case I, the performance is 0.382 in the Naïve approach, which is much slower compared to either multiprocessing or multithreading.

In the Multiprocessing approach, performance improves as the number of child processes increases, but this improvement starts to level off after 8 child processes. At 16 and 20 child processes, the performance begins to stagnate or even slightly decline. For instance, in Case I, the performance for 2 child processes is 0.382, but it decreases to 1.039 with 20 child processes. Other cases follow a similar trend, where performance improves up to 8 processes, and beyond that, the improvement diminishes.

On the other hand, the Multithreading approach shows a more consistent improvement in performance as the number of threads increases, especially in the lower ranges (from 2 to 8 threads). For Case I, performance improves from 0.382 in the Naïve approach to 0.869 for 20 threads. This trend continues across other cases, demonstrating that the threading model utilizes system resources more effectively compared to multiprocessing in these experiments.

Overall, Multithreading offers better performance gains compared to Multiprocessing, especially when moving from fewer to more threads. After a certain point, Multiprocessing shows diminishing returns, making Multithreading more efficient in this scenario. Meanwhile, the Naïve approach remains the slowest, with significantly poorer performance than both multiprocessing and multithreading.

Conclusion

In conclusion, the results of this project clearly demonstrate the significant impact that parallel programming techniques can have on the performance of data-intensive applications, such as the sales-analysis program used in this study. The naïve sequential approach provided a baseline for comparison, yet it showed the slowest execution time due to its reliance on a single processing path. In contrast, both multiprocessing and multithreading achieved substantial improvements by distributing the workload across multiple execution units. The experiments also revealed important differences between the two parallel strategies: multiprocessing benefited from having completely separate memory spaces for each process, while multithreading faced more contention due to shared memory. Furthermore, the findings highlighted that increasing the number of processes or threads beyond the hardware's capabilities—especially on a device with only 4 CPU cores—leads to oversubscription, excessive context switching, and a decline in performance. Overall, this project emphasizes the importance of carefully selecting the appropriate level and method of parallelism to balance speed, efficiency, and resource utilization. It also shows that optimal results are achieved when the degree of parallelism matches the limitations and strengths of the underlying hardware.